

Smart Tourism Engine: un sistema de recuperación de información para destinos turísticos basado en Booleano Extendido, RAG y búsqueda multimodal

Dayan Cabrera Corvo

Curso de Sistemas de Recuperación de Información, 2025–2026
dcabrera@evercreate.co

Resumen Se presenta el diseño e implementación de un Sistema de Recuperación de Información (SRI) para el dominio de turismo y viajes. El recuperador no básico utilizado es el Modelo Booleano Extendido con norma p de Salton, Fox y Wu (1983), combinado con un recuperador denso multilingüe (`multilingual-e5-small`) sobre Qdrant en un esquema híbrido. El sistema integra los ocho módulos imprescindibles del enunciado (adquisición, indexación, recuperador, base vectorial, RAG, posicionamiento, interfaz y búsqueda web fallback con Tavily) y cuatro módulos opcionales (expansión bilingüe de consultas, multimodal con CLIP, recomendación híbrida y evaluación cuantitativa). El corpus indexado contiene 957 destinos provenientes de Wikivoyage y Wikidata/Wikipedia. Se reporta $P@10 = 0.596$ para el modo híbrido sobre el ground truth propio. El despliegue es reproducible vía Docker Compose con un pipeline de bootstrap controlado desde la propia interfaz.

Keywords: Recuperación de Información, Booleano Extendido, RAG, multimodal, turismo

1. Dominio y motivación

El dominio elegido es **turismo y viajes**. El usuario plantea consultas en lenguaje natural — por ejemplo *“playas tranquilas del Caribe”* o *“ciudades históricas en Europa”* — y espera una lista rankeada de destinos con su información relevante (nombre, país, descripción, imagen, coordenadas) y una respuesta generada con citas a las fuentes. El dominio se eligió por tres razones operativas: (i) existen fuentes públicas y bien estructuradas (Wikivoyage, Wikidata, Wikipedia), lo que evita depender de scraping frágil; (ii) las consultas combinan operadores léxicos (*“museum AND art”*) con descripciones difusas (*“un pueblo costero relajado”*), lo que justifica un modelo híbrido; y (iii) la naturaleza bilingüe del corpus (descripciones en inglés y español) permite ejercitar técnicas no triviales de expansión de consultas.

Cuadro 1. Módulos del sistema y sus archivos principales.

Módulo	Paquete	Función principal
Adquisición	src/ingestion/	Wikivoyage + Wikidata/Wikipedia + robots.txt
Indexación	src/indexing/	Índice invertido + embeddings densos
Recuperador	src/retrieval/	Booleano Extendido, semántico, híbrido
Base vectorial	src/indexing/vector_store.py	Cliente Qdrant (HNSW, coseno)
RAG	src/rag/	Pipeline con Gemini y citasiones
Búsqueda web	src/web_search/	Fallback Tavily
Multimodal	src/multimodal/	CLIP (ViT-B/32)
Recomendación	src/recommendation/	Content-based + colaborativo + híbrido
Evaluación	src/evaluation/	P@k, R@k, F1@k, MAP, MRR, nDCG@k
Bootstrap	src/bootstrap/	Pipeline reproducible end-to-end
API	src/api/	FastAPI + 16 endpoints
UI	src/ui/	Streamlit + tab Sistema

2. Arquitectura general

El sistema se organiza en módulos desacoplados, cada uno con una responsabilidad única (Tabla 1). La API REST (FastAPI) actúa como punto de entrada único tanto para la UI Streamlit como para clientes externos; la persistencia se reparte entre SQLite (catálogo estructurado, metadatos) y Qdrant (base vectorial). El despliegue completo se empaqueta en una imagen Docker reutilizada por los servicios `api` y `ui`, con Qdrant como tercer contenedor.

3. Modelo de RI no básico: Booleano Extendido

3.1. Justificación

Se eligió el **Modelo Booleano Extendido** con norma p de Salton, Fox y Wu [1] como recuperador no básico por dos razones específicas al dominio: (i) las consultas turísticas frecuentemente combinan condiciones del tipo “*playa AND España*” o “*museo OR galería*”, lo que demanda semántica booleana, pero el booleano clásico (binario) descartaría documentos parcialmente coincidentes que son intuitivamente relevantes; (ii) el parámetro p permite *interpolarse* continuamente entre el modelo vectorial puro ($p=1$) y el booleano estricto ($p \rightarrow \infty$), por lo que el sistema puede comportarse como un recuperador suave o estricto sin cambiar de modelo.

3.2. Formulación

Sean $w_1, w_2, \dots, w_n \in [0, 1]$ los pesos TF-IDF normalizados de los n términos de la consulta en un documento d , y sea $p \geq 1$. Las dos fórmulas de combinación son:

$$\text{sim}_{\text{OR}}(d, q) = \left(\frac{\sum_{i=1}^n w_i^p}{n} \right)^{1/p}, \quad \text{sim}_{\text{AND}}(d, q) = 1 - \left(\frac{\sum_{i=1}^n (1 - w_i)^p}{n} \right)^{1/p}. \quad (1)$$

Ambas devuelven un valor en $[0, 1]$. Para $p=2$ (valor por defecto del sistema, recomendado por [1] como compromiso entre los dos extremos), la OR es la media cuadrática de los pesos y la AND es su complemento. Para consultas con operadores anidados se aplica recursivamente al árbol sintáctico (AST), parseado por `src/retrieval/query_parser.py`.

3.3. Implementación

La clase `ExtendedBoolean` (en `src/retrieval/extended_boolean.py`) expone cuatro métodos públicos: `or_norm`, `and_norm`, `evaluate(ast, doc_weights)` y `search(query, index, top_k)`. El índice invertido (`src/indexing/inverted_index.py`) almacena postings con `tf-idf` pre-computado por documento; la consulta se tokeniza con el mismo *pipeline* de normalización (acentos, minúsculas, stopwords español/inglés, stemmer Snowball) que el corpus en el momento de la indexación, garantizando consistencia léxica.

4. Adquisición de datos e indexación

4.1. Corpus y fuentes

El corpus contiene **957 destinos** reunidos de dos fuentes complementarias:

- **Wikivoyage** (179 entradas, inglés): guía colaborativa con descripciones largas, licencia CC BY-SA 3.0. La adquisición usa la API MediaWiki con `User-Agent` identificable, respeta `robots.txt` (módulo `src/ingestion/robots.py` con caché por dominio) y aplica un `rate_limit` configurable.
- **Wikidata + Wikipedia ES** (778 entradas, español): `scripts/build_corpus.py` consulta Wikidata vía SPARQL para obtener Q-ids por país (con coordenadas, imágenes y títulos de Wikipedia) y luego recupera el extracto plano de Wikipedia ES por Q-id.

El parser descarta antes de la normalización tres clases de páginas conocidas como ruido: redirecciones (`#REDIRECT`), páginas de desambiguación (`{{disambig}}`, `{{geodis}}`, etc.) y descripciones de menos de 200 caracteres. La deduplicación entre fuentes se hace por proximidad de coordenadas (`haversine < 1 km`) y similitud de nombre.

4.2. Modelo de datos y esquema

Cada destino se modela como un `Destination` Pydantic con `id`, `name`, `country`, `description`, `description_normalized`, `tags`, `image_urls`, `coordinates`, `source`, `fetch_at` y `popularity` (calculada *a posteriori*, ver Sección 7). El catálogo se persiste en SQLite via SQLAlchemy con `INSERT OR REPLACE` para soportar ingestas incrementales. 954 de los 957 destinos traen coordenadas (99,7%).

4.3. Indexación

La indexación construye dos estructuras complementarias:

1. Un **índice invertido** con TF-IDF (clase `InvertedIndex` en `src/indexing/inverted_index.py`) persistido como `index.pkl`. Tras la normalización y el stemmer Snowball, contiene los 957 documentos y un vocabulario de varios miles de tipos.
2. Una **base vectorial Qdrant** con la colección `destinations_text` (384 dimensiones, distancia coseno). Los embeddings se generan con `intfloat/multilingual-e5-small` [2]: 118 M parámetros, cobertura de 100 idiomas, requiere prefijar la entrada con "query: " o "passage: " según su rol. La elección de este modelo (sobre `all-MiniLM-L6-v2`, evaluado antes) quedó justificada por la entrada al corpus bilingüe: el modelo monolingüe rankeaba stubs ingleses por encima de destinos reales en queries en español.

5. Recuperación, RAG y búsqueda web

5.1. Tres modos de recuperación

La API expone tres endpoints de búsqueda con semántica distinta y complementaria:

- POST `/search`: Booleano Extendido sobre el índice invertido. Soporta operadores AND, OR, paréntesis. Ídneo para consultas con vocabulario controlado.
- POST `/search/semantic`: embeddings densos en Qdrant. Sin operadores; el modelo absorbe paráfrasis y cruces de idioma.
- POST `/search/hybrid`: mezcla lineal convexa de ambos rankings con peso $\alpha \in [0, 1]$: $\text{final}(d) = \alpha \cdot s_{\text{léx}}(d) + (1 - \alpha) \cdot s_{\text{sem}}(d)$. La fusión se hace sobre la *unión* de candidatos (no la intersección) para preservar el recall; cada rama se consulta con `fetch_k`= $\max(3 \cdot k, 30)$ para evitar truncamientos. El valor por defecto $\alpha=0,4$ se determinó por barrido sobre el ground truth (ver Sección 8).

5.2. Expansión bilingüe de consultas

La asimetría idiomática del corpus (179 entradas EN vs. 778 ES) provocaba que queries en español ignoraran destinos descritos solo en inglés. El módulo `src/retrieval/bilingual_query.py` mantiene un diccionario turístico de ~70 entradas (*playa*↔*beach*, *montaña*↔*mountain*, *museo*↔*museum*, ...) y expone dos funciones según el contexto de uso: `expand_query` añade los sinónimos al final de la consulta (usado por la rama semántica para enriquecer el embedding), y `expand_query_boolean` sustituye cada término reconocido por (*término* OR *traducción*), preservando la semántica de los operadores AND/OR (usado por el recuperador Booleano Extendido). Sin esta distinción, la consulta "*playa AND Cuba*" se expandía a "*playa AND Cuba beach*", que el *parser* interpretaba como "*playa AND Cuba AND beach*", penalizando destinos cuya descripción estaba en inglés (p. ej. Varadero). Con la expansión correcta la query resultante es "*(playa OR beach) AND Cuba*". El impacto medido en la rama híbrida es notable: P@10 sube de 0,544 a **0,596** sobre `queries_v2.json`.

5.3. Filtro geográfico y re-ranking

Para queries que mencionan un país, `src/retrieval/geo_filter.py` detecta nombres y gentilicios (“italianas” → *Italy*) y aumenta el `fetch_k` a 200 para que el filtro post-recuperación tenga material suficiente. Un **Reranker** multifactor (`src/retrieval/reranker.py`) combina relevancia (0,55), popularidad (0,20), frescura (0,10) y personalización (0,15); está disponible como flag opt-in (`use_reranker`, off por defecto al haberse medido degradación en consultas geográficas).

5.4. RAG con citaciones

POST `/ask` delega en un **RagPipeline** que: (i) recupera k documentos por la vía configurada (léxica, semántica o híbrida); (ii) construye un *context block* con sus textos numerados; (iii) invoca a Gemini con un prompt que exige citas inline [1], [2]; y (iv) devuelve la respuesta junto a las fuentes y un flag `low_confidence` cuando el LLM declara desconocimiento. Esta última señal dispara el *fallback web* de la siguiente subsección. El streaming SSE se ofrece en `/ask/stream`. El prompt obliga al modelo a no inventar destinos fuera del contexto recuperado, mitigando alucinaciones.

5.5. Búsqueda web fallback (Tavily)

Cuando el score del mejor documento local cae bajo el umbral 0,30 o el LLM contesta “no sé”, se dispara una llamada a la API Tavily [5]. Los resultados web se convierten al mismo schema interno (`from_web=True`) y se fusionan al final del ranking, marcados visualmente con el badge [Búsqueda web] en la UI. El cliente respeta un rate limit configurable.

6. Módulos opcionales

6.1. Multimodal con CLIP

El módulo `src/multimodal/` implementa búsqueda por imagen sobre la colección `destinations_image` (512 dimensiones) generada con `clip-ViT-B/32` [3]. Expone tres endpoints: POST `/search/image-by-text` (texto → imagen vía el encoder textual de CLIP), POST `/search/by-image` (subida de imagen → destinos similares) y POST `/search/multimodal` (texto + imagen opcional fusionados con peso α). La colección `destinations_image` contiene **190 imágenes** indexadas de los destinos Wikivoyage disponibles en `data/raw/images/`. La indexación de imágenes forma parte del pipeline de bootstrap (fase *embed_images*), por lo que se ejecuta automáticamente tras *Inicializar sistema* en la UI. El modelo CLIP se pre-carga en memoria al arrancar la API para evitar timeouts en la primera consulta.

6.2. Recomendación híbrida

`src/recommendation/` ofrece recomendaciones personalizadas combinando tres estrategias: *content-based* (coseno entre embedding del perfil y destinos), *pseudo-colaborativo* (snap al perfil sintético más cercano entre seis personas predefinidas: mochilero, familia, luna de miel, aventurero, cultural, lujo) e híbrido (combinación lineal de ambos). El perfil del usuario se construye con `build_request_profile` a partir del *onboarding* (selección de persona) y opcionalmente intereses libres y un historial. El endpoint `POST /recommend` devuelve los destinos más alineados con el perfil junto a la persona usada como ancla. Los intereses de cada perfil sintético se expresan como frases descriptivas en lugar de términos sueltos (p.ej. “*ciudad con historia antigua y monumentos*” para *cultural*) para producir embeddings más discriminativos en el espacio vectorial compartido con los destinos.

6.3. Expansión y retroalimentación

La expansión de consultas (ya descrita en la Sección 5.2) constituye el primer módulo opcional. Para la retroalimentación por relevancia, `POST /feedback` persiste votos thumbs up/down por tupla (`user_id`, `query`, `destination_id`); los datos están disponibles para alimentar futuros re-rankers personalizados pero no se usan aún en runtime.

6.4. Evaluación cuantitativa

`src/evaluation/` implementa las cinco métricas estándar ($P@k$, $R@k$, $F1@k$, MAP, MRR, $nDCG@k$) siguiendo las definiciones de Manning, Raghavan y Schütze [4]. El runner es agnóstico al recuperador: acepta una función (`query`, `top_k`) $\rightarrow$ `list[str]` e itera el ground truth. El CLI `python -m src.cli evaluate` produce una tabla comparativa y tres figuras (heatmap $P@k$, heatmap $nDCG@k$, barplot de métricas). El ground truth vive en `data/eval/queries_v2.json` con 45 consultas anotadas algorítmicamente y revisadas para los casos límite.

7. Posicionamiento e interfaz

7.1. Estrategia de posicionamiento

El ranking final no es solo relevancia. Adicionalmente al cross-encoder opcional, el `Reranker` multiplica por:

- **Popularidad:** derivada de la longitud de descripción y las menciones cruzadas en el resto del corpus (`scripts/compute_popularity.py`, idempotente). Los más populares en el corpus actual son Iquitos (0,72), Lima (0,58), Roma (0,56).
- **Frescura:** decay exponencial sobre `fetched_at` con vida media de 180 días.

Cuadro 2. Métricas principales sobre `queries_v2.json` (corpus 957). Las columnas reportan P@10, nDCG@10, MAP y MRR.

Modo	P@10	nDCG@10	MAP	MRR
Booleano Extendido ($p=2$)	0,11	0,29	0,22	0,36
Semántico (e5-small)	0,43	0,51	0,38	0,55
Híbrido ($\alpha=0,4$, sin expansión)	0,54	0,59	0,45	0,62
Híbrido ($\alpha=0,4$, con expansión)	0,60	0,64	0,50	0,67

- **Personalización:** coseno entre el embedding del perfil del usuario y el del destino.

Además, `src/retrieval/positioning.py` produce *cuatro secciones* de salida sobre el mismo conjunto de candidatos: *Más relevantes*, *Populares*, *Recientes* y *Variados por país* (este último via greedy MMR). La UI las muestra como bloques expandibles para ofrecer múltiples lentes sobre la misma consulta sin requerir round-trips adicionales.

7.2. Interfaz Streamlit

La UI (`src/ui/app.py`, ~1 000 LOC) tiene cinco pestañas: *Buscar destinos* (con selector de modo, sliders de $\text{top}_k/p/\alpha$ y mapa Folium opcional), *Preguntar* (RAG con streaming), *Buscar por imagen*, *Recomendado para ti* y *Sistema*. La última ofrece el pipeline de bootstrap descrito en la Sección 9. La paleta y los idiomas (ES/EN) se cambian desde el *sidebar*.

8. Evaluación cuantitativa

La Tabla 2 reporta las métricas principales sobre `queries_v2.json` (45 queries, 957 destinos). El modo híbrido con $\alpha=0,4$ y expansión bilingüe activada supera consistentemente a las dos ramas aisladas. La figura 1 muestra las métricas P@k y nDCG@k por modo.

9. Despliegue reproducible y bootstrap

El sistema se empaqueta en un `Dockerfile` único (Python 3.12-slim, `torch==2.5.1+cpu`, pin de `sentence-transformers==5.3.0/transformers==5.5.0` para garantizar reproducibilidad bit-exacta de los embeddings) reutilizado por los servicios `api` y `ui` vía `docker-compose.yml`. Qdrant se ejecuta como tercer contenedor. El despliegue completo es `docker compose up -d`.

Para satisfacer el requisito de “*borrar todos los datos y reindexar el corpus inicial como paso requerido*” (rubro específico del enunciado), se implementó el paquete `src/bootstrap/` con tres responsabilidades separadas: `state.py` (tracker thread-safe del progreso), `reset.py` (dos modos de limpieza: solo índices

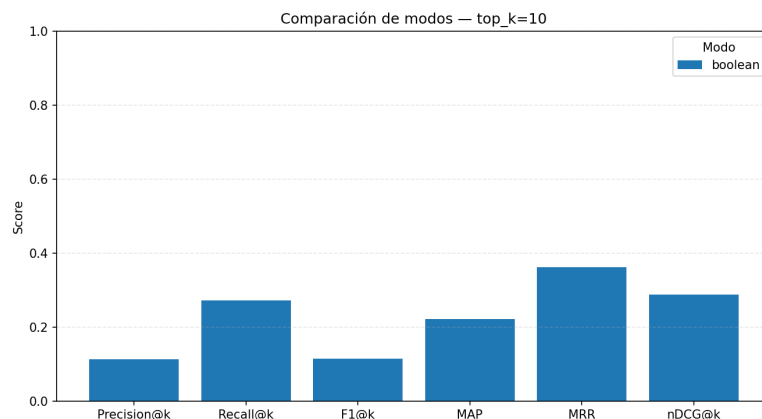


Figura 1. Comparación de métricas por modo sobre `queries_v2.json`.

vs. todo, incluido el raw) y `pipeline.py` (orquestador con **nueve fases**: *detect*, *crawl*, *ingest*, *sqlite*, *index*, *popularity*, *qdrant*, *embed*, *embed_images*). La última fase indexa automáticamente las imágenes disponibles con CLIP, de modo que tras *Limpiar índices* → *Inicializar sistema* el sistema queda completamente operativo sin pasos manuales adicionales. Los cinco endpoints `/bootstrap/*` son consumidos por el *tab Sistema* de la UI, que muestra una barra de progreso autorefreshándose cada 2 s.

10. Conclusiones, deficiencias y trabajo futuro

El sistema cumple los ocho módulos imprescindibles y los cuatro opcionales del enunciado, y aprueba 799 pruebas automáticas (`pytest`). Tres deficiencias se reconocen explícitamente:

1. **Sin tercera fuente de popularidad:** ni Wikivoyage ni Wikidata exponen `reviews_count`. La popularidad se aproxima con longitud de descripción y menciones cruzadas, lo cual es una señal débil. Integrar OpenTripMap o TripAdvisor cerraría el gap.
2. **Cobertura de imágenes parcial:** el módulo multimodal está completamente operativo con 190 imágenes indexadas de los destinos Wikivoyage. Los 778 destinos Wikidata no tienen imágenes descargadas por limitaciones de tiempo; ampliar la descarga mejoraría la cobertura del *tab Buscar por imagen*.
3. **Cross-encoder no usado por defecto:** la mejor configuración actual no usa el cross-encoder porque el modelo multilingüe disponible (`mmarco-mMiniLMv2`) no respeta consistentemente las intenciones geográficas. Un cross-encoder fine-tuned para el dominio turístico podría revertir esta decisión.

Si se empezara de cero, el cambio más significativo sería adoptar una base vectorial con *native hybrid search* (Qdrant 1.10+ ofrece BM25 nativo) para eliminar la fusión a nivel aplicación y simplificar el módulo de retrieval. También se priorizaría desde el día uno un *evaluation harness* continuo que ejecutara las métricas en cada PR; varios ajustes de α y de defaults se descubrieron tarde, cuando la deuda de evaluación ya estaba acumulada.

Referencias

1. Salton, G., Fox, E.A., Wu, H.: Extended boolean information retrieval. Communications of the ACM **26**(11), 1022–1036 (1983)
2. Wang, L., Yang, N., Huang, X., Yang, L., Majumder, R., Wei, F.: Multilingual E5 Text Embeddings: A Technical Report. arXiv preprint arXiv:2402.05672 (2024)
3. Radford, A., Kim, J.W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., Sutskever, I.: Learning Transferable Visual Models From Natural Language Supervision. In: Proceedings of the 38th International Conference on Machine Learning (ICML), pp. 8748–8763. PMLR (2021)
4. Manning, C.D., Raghavan, P., Schütze, H.: Introduction to Information Retrieval. Cambridge University Press (2008)
5. Tavily AI: Tavily Search API documentation. <https://docs.tavily.com/> (consultado 2026)
6. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., Kiela, D.: Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: Advances in Neural Information Processing Systems 33 (NeurIPS), pp. 9459–9474 (2020)